

Robust Autonomous Driving via Model Predictive Path Integral Control

Henry Chang

William Howe

Abstract

Autonomous driving promises major societal benefits, including reduced traffic fatalities—70% of which are caused by human error—as well as lower fuel consumption and decreased urban parking demand. While current systems perform well in controlled settings, real-world driving remains challenging due to uncertain dynamics and the need to anticipate other agents’ behavior. We address these issues with Model Predictive Path Integral (MPPI) control, combined with Control Barrier Functions (CBF), enabling robust and responsive trajectory planning in complex environments.

1. Introduction

Autonomous driving is one of the most transformative developments in modern transportation, attracting over \$208 billion in investment in 2023 alone [10]. Beyond its economic impact, it holds promise for significant societal benefits. Car accidents are currently the 12th leading cause of death globally—and the leading cause for ages 5–29—with around 70% of fatalities linked to human error [1, 8]. By removing the human driver, autonomous systems have the potential to drastically improve road safety.

Environmental and urban advantages also follow: autonomous vehicles can reduce fuel consumption through optimized control [13], and shared fleets may ease parking demand in dense cities [2].

While autonomous systems have shown strong performance in controlled settings, such as in the DARPA Grand and Urban Challenges [11, 12], real-world environments remain challenging—especially near the limits of handling, where control depends on difficult-to-predict factors like tire-road friction [14]. Additionally, many planning methods fail to account for the behavior of other agents, leading to overly cautious outcomes such as the “frozen robot” problem [7].

To overcome these limitations, we propose a framework that combines the Control Barrier Function and game-theoretic modeling of agent interactions with Model Predictive Path Integral (MPPI) control. This allows autonomous

agents to generate responsive, high-performance trajectories that not only are robust in static obstacle avoidance, but also account for system dynamics and the strategic behavior of other dynamic road users.

2. Related work

2.1. MPPI

Model Predictive Path Integral (MPPI) control is a sampling-based, derivative-free optimization technique within the broader family of Model Predictive Control (MPC) methods. Unlike conventional MPC approaches, MPPI does not require gradient information or approximations of the objective function or system dynamics, making it especially effective for handling highly dynamic systems such as autonomous urban navigation and highway lane merging. Its parallel sampling strategy leverages the computational power of modern GPUs to enable real-time trajectory optimization [9]. However, MPPI’s performance is sensitive to the choice of input distribution parameters, such as the mean and covariance, which can lead to a majority of rollouts violating system constraints if poorly configured. To mitigate this, recent advancements such as output-sampled MPPI (oMPPI) have been proposed, improving the feasibility of sampled trajectories with respect to output constraints [15]. This makes MPPI a powerful and increasingly practical tool for real-world autonomous control applications.

2.2. MPPI with Control Barrier Function

To improve the robustness of MPPI in environments with static obstacles, recent work has explored incorporating Control Barrier Functions (CBFs) [3] into the sampling process. CBFs offer a principled way to enforce safety constraints by defining a forward-invariant set. Essentially, keeping the system out of unsafe regions by construction.

The Shield MPPI method [16] modifies the MPPI cost by adding a CBF-based penalty that discourages sampled trajectories from violating safety margins. This soft constraint approach avoids the need to solve constrained optimization problems and maintains MPPI’s ability to scale efficiently with parallel rollouts. In their experiments with

a unicycle model, the method showed strong safety performance even with a limited number of samples, successfully avoiding collisions in cluttered, static environments. Figure 1 illustrates how MPPI handles both static and dynamic obstacles with CBF.

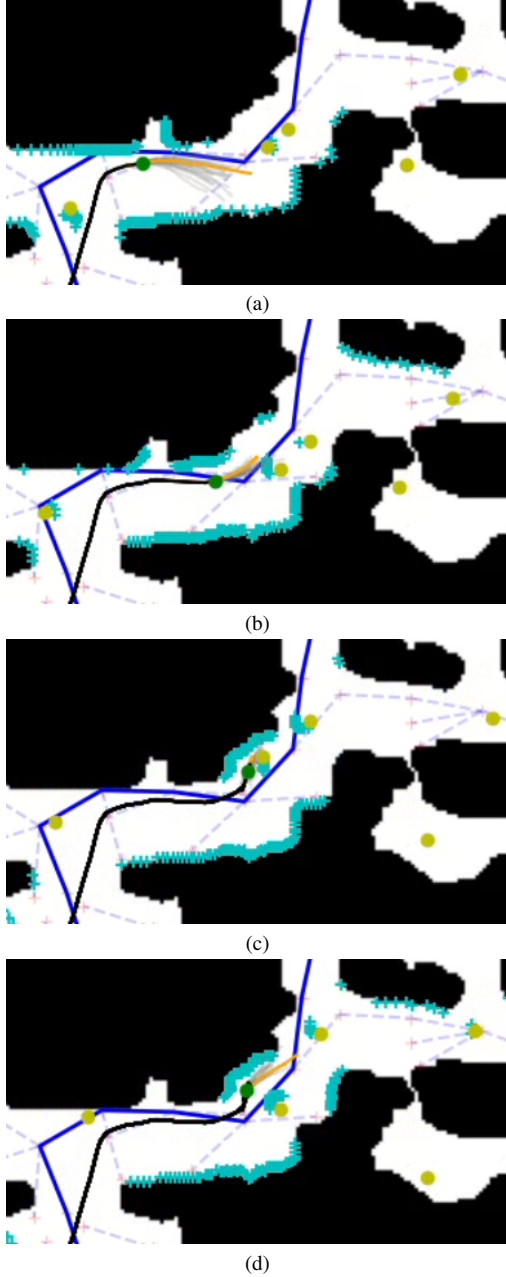


Figure 1. Dynamic obstacle avoidance sequence. As the agent progresses in the sequence from (a)-(d), the obstacle proceeds to move into the agents way. The CBF related to the obstacle drives the sampled sampled trajectories to decrease in length to avoid collision. Once the obstacle moves away, the trajectories extend out again and the agent resume following the path.

2.3. RRT*

The Rapidly-exploring Random Tree (RRT) algorithm is a sampling-based motion planning method designed to efficiently explore high-dimensional spaces [6]. It incrementally builds a tree by randomly sampling points in the space and extending the tree toward each sample from the nearest existing node, subject to system constraints. RRT is particularly effective in complex, cluttered environments because it rapidly expands into unexplored areas, making it well-suited for path planning in robotics and autonomous navigation. However, while RRT can find feasible paths quickly, these paths are often suboptimal and may require post-processing to improve smoothness and safety.

Rapidly-exploring Random Tree Star (RRT*) is an advanced sampling-based algorithm used for motion planning that incrementally builds a tree rooted at the start state to efficiently explore the search space. Unlike the basic RRT, RRT* improves path optimality by rewiring the tree as new samples are added, gradually converging towards the shortest or lowest-cost path to the goal [4]. This makes RRT* well-suited for finding feasible and near-optimal trajectories in high-dimensional and complex environments.

3. Methodology

This project explores the MPPI framework for motion planning in mobile robots, with the long-term goal of developing a robust, publishable system for real-time autonomous navigation. The first phase focuses on integrating Control Barrier Functions (CBFs) with MPPI to ensure safe navigation around static obstacles. A key milestone will be demonstrating collision-free trajectories in a mapped environment.

The second phase tackles dynamic obstacles using game-theoretic modeling, where agents reason about others' behaviors to avoid collisions in shared spaces. Example scenarios include multi-robot coordination or highway-style lane merging. To support long-horizon planning, a global planner like RRT* will be used to guide the robot's local MPPI controller.

Finally, we will explore sampling improvements such as output-sampled MPPI (oMPPI) to further improve computational efficiency and responsiveness. This simulation framework will lay the foundation for future real-world deployment and multi-agent extensions.

3.1. Model Predictive Path Integral

Consider a deterministic, discrete, nonlinear system

$$x_{k+1} = f(x_k, u_k) \quad (1)$$

where $x_k \in \mathcal{D} \subseteq \mathbb{R}^{n_x}$ is the state and $u_k \in \mathbb{R}^{n_u}$ is the control input at time step k . In the MPPI setup, the controlled system is stochastic, where the system is given some mean

control $v_k \in \mathbb{R}^{n_u}$ and covariance matrix $\Sigma_\epsilon \in \mathbb{R}^{n_u \times n_u}$. Consequently, the actual control follows a Gaussian distribution $u_k \sim \mathcal{N}(0, \Sigma_\epsilon)$, and the sequence $\mathbf{u} = (u_0, \dots, u_{K-1})$ becomes multi-modal distributed. When sample M different input sequences $\mathbf{u}$ with K steps, a selection criterion based on the output trajectory (or an objective function) is given by

$$S(\mathbf{u}) = \phi(x_K) + \sum_{k=0}^{K-1} q(x_k) + \lambda \sum_{k=0}^{K-1} v_k^T \Sigma_\epsilon^{-1} u_k, \quad (2)$$

where $q(x_k)$ and $\phi(x_k)$ denote the state-dependent running cost and terminal cost, respectively. The hyperparameter λ controls how selectively the MPPI algorithm weighs the sampled trajectories. Here, we denote the cost of the m^{th} sampled trajectory by S^m , as a shorthand for $S(\mathbf{u}^m)$. Its value, computed according to Eq. (2), is given by:

$$S^m = \phi(x_K^m) + \sum_{k=0}^{K-1} q(x_k^m) + \lambda (v_k^m)^T \Sigma_\epsilon^{-1} (v_k^m + \epsilon_k^m), \quad (3)$$

3.2. Control Barrier Function

For implementation, the Discrete-time Control Barrier Function (DCBF) is defined as follows. Let $h : \mathbb{R}^n \rightarrow \mathbb{R}$ be a Lipschitz continuous function, and define the safe set $\mathcal{S} \subseteq \mathcal{D}$ as

$$\mathcal{S} := \{x \in \mathcal{D} \mid h(x) \geq 0\}. \quad (4)$$

Let $\mathcal{U}$ denote the set of admissible controls. The function h is a DCBF for the nonlinear system (1) if, for all $x \in \mathcal{D}$, there exists a control input $u \in \mathcal{U}$ such that

$$h(f(x, u)) - h(x) \geq -p(h(x)), \quad (5)$$

where $p : \mathbb{R} \rightarrow \mathbb{R}$ is a class- κ function. In this work, we adopt the specific form $p(r) = \beta r$ with $\beta \in (0, 1)$. The DCBF framework exhibits two key theoretical properties, as discussed in [16]:

Property 1 (Forward Invariance) *Given an initial condition $x_0 \in \mathcal{S}$ and a control sequence $\{v_k\}_{k=0}^\infty$ such that all (x_k, v_k) pairs satisfy the DCBF condition (5), then $x_k \in \mathcal{S}$ for all $k \in \mathbb{Z}_{\geq 0}$.*

Property 2 (Asymptotic Safety Recovery) *Let $x_0 \in \mathcal{D} \setminus \mathcal{S}$ and a control sequence $\{v_k\}_{k=0}^\infty$ such that, for all $k \in \mathbb{Z}_{\geq 0}$, the pair (x_k, v_k) satisfies the DCBF condition. Then, the state x_k converges asymptotically to the safe set $\mathcal{S}$.*

To maintain safety while reducing computational load from long planning horizons, we augment the cost function with a DCBF term. This addition allows the controller to assess the safety of actions by considering only a limited number of future steps. Specifically, a state-dependent barrier function

term is included in the standard MPPI sampling costs. Let $\alpha = 1 - \beta \in (0, 1)$, the DCBF constraints violation penalty cost is defined as,

$$C_{cbf}(x_k, x_{k-1}) = C \max\{-h(x_k) + \alpha h(x_{k-1}), 0\}, \quad (6)$$

where C is a tuning parameter that scales the penalty cost based on the degree of constraint violation. Lastly, the cost of the m^{th} sampled trajectory S^m in (3) is modified as,

$$\tilde{S}^m = S^m + \sum_{k=0}^K C_{cbf}(x_k^m, x_{k-1}^m) = S^m + \sum_{k=0}^K C_{cbf}(z_k^m), \quad (7)$$

3.3. Robust MPPI Algorithm

The proposed robust-MPPI algorithm is outlined in Algorithm 1. Line 2 retrieves the current state estimate x_0 . Lines 3–13 handle trajectory sampling and cost evaluation: initial states are set (Line 4), control noise ϵ^m is sampled (Line 5), and noisy inputs $u_k^m = v_k + \epsilon_k^m$ are used to propagate the system state (Line 8). The modified trajectory cost $\tilde{S}^m$, including the DCBF penalty, is accumulated at each step (Lines 10 and 12). The updated control sequence $\mathbf{v}^+$ is computed in Line 14 using the optimal control update rule. Safe controls are then executed (Line 15), and $\mathbf{v}^+$ is carried forward to warm-start the next iteration (Line 16).

Algorithm 1 Robust MPPI Algorithm

Require: MPPI costs $q(\cdot), \phi(\cdot)$, parameters γ, Σ_ϵ ; Initial control sequence $\mathbf{v}$

- 1: **while** task not complete **do**
- 2: $x_0 \leftarrow \text{GetStateEstimate}()$
- 3: **for** $m \leftarrow 0$ to $M - 1$ **in parallel** **do**
- 4: $x_0^m \leftarrow x_0, z_0^m \leftarrow [x_0^T, x_0^T]^T, \tilde{S}^m \leftarrow 0$
- 5: Sample $\epsilon^m \leftarrow \{\epsilon_0^m, \dots, \epsilon_{K-1}^m\}$
- 6: **for** $k \leftarrow 0$ to $K - 1$ **do**
- 7: $u_k^m \leftarrow v_k + \epsilon_k^m$
- 8: $x_{k+1}^m \leftarrow f(x_k^m, u_k^m)$
- 9: $z_{k+1}^m \leftarrow [(x_{k+1}^m)^T, (x_k^m)^T]^T$
- 10: $\tilde{S}^m \leftarrow \tilde{S}^m + q(x_k^m) + \gamma v_k^T \Sigma_\epsilon^{-1} u_k^m + C_{cbf}(z_k^m)$
- 11: **end for**
- 12: $\tilde{S}^m \leftarrow \tilde{S}^m + \phi(x_K^m) + C_{cbf}(z_K^m)$
- 13: **end for**
- 14: $\mathbf{v}^+ \leftarrow \text{OptimalControl}(\{\tilde{S}^m\}_{m=0}^{M-1}, \{\mathbf{u}^m\}_{m=0}^{M-1})$
- 15: ExecuteCommand($\mathbf{v}_0^+$)
- 16: $\mathbf{v} \leftarrow \mathbf{v}^+$
- 17: **end while**

4. Experimental setup and results

This section presents the implementation results of a unicycle robot running an MPPI controller with local obstacle

avoidance. Then, incorporate the controller with an RRT* global planner.

4.1. MPPI with CBF

Here we test the usage of MPPI with a CBF on static obstacles. Objects are placed between the agent and a final goal state and then the agent must navigate from the initial position to the goal. The agent moves based on standard unicycle dynamics. Figure 3 illustrates a single test run. The unicycle robot started at position (0,0) and the navigation goal is (5,5). The controller randomly samples $K = 300$ roll-outs with a horizon $H = 40$ steps in each step. The test consisted of 50 trials with random initial orientation, and 95% of the results are collision-free. With more sampled roll-outs $K = 500$, the obstacle avoidance rate increases to 98%.

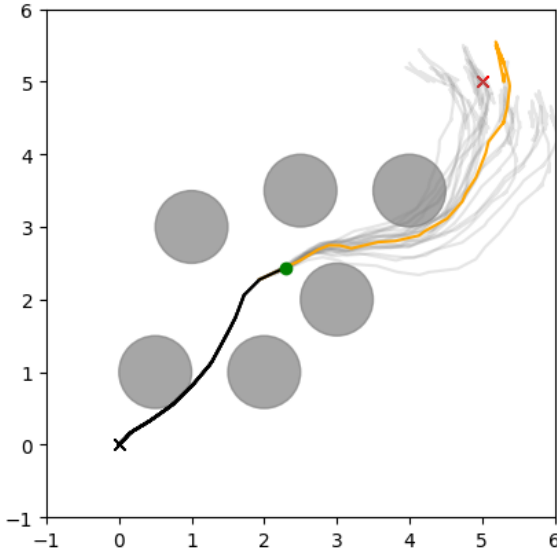


Figure 2. Caption

4.2. RRT* global planner

For full-scale application, we apply MPPI with CBF to a trajectory-following task. The trajectory is generated using RRT* based on specified start and goal points within pre-generated maps. Once the trajectory is generated, the agent follows this path toward the goal while avoiding detected obstacles using a Control Barrier Function (CBF). Figure 3 illustrates an example of a single trajectory run. The solid blue line represents the path produced by RRT*, while the black line shows the actual trajectory taken by the agent. The faint blue dashed lines correspond to the remaining sampled points from RRT*. The agent must follow the planned path while avoiding walls (black areas) and dynamic obstacles, which are marked by yellow circles.

The MPPI controller tracks the path using a terminal cost

associated with a moving waypoint. At each time step, a waypoint is selected a certain number of steps ahead of the agent's nearest position on the path. This number of steps is determined by the trajectory horizon and a specified "look-ahead ratio," defined as the ratio of the waypoint offset to the trajectory length. This parameter is expressed as $\frac{l_{\text{waypoint}}}{l_{\text{traj}}}$ in Table 1.

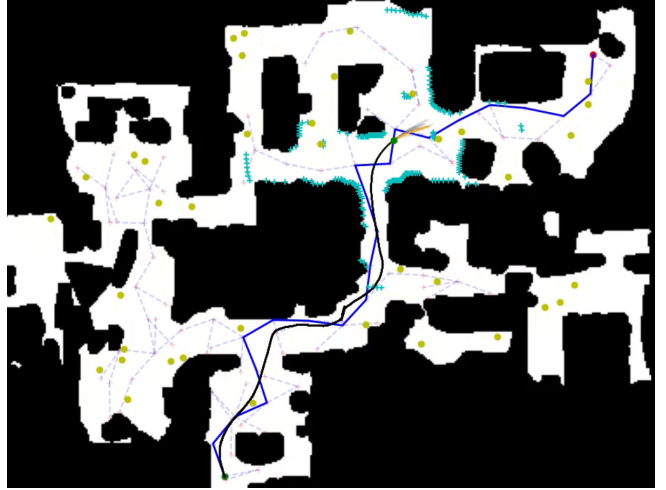


Figure 3. Simulation example for MPPI applied to trajectory following with CBF-based obstacle avoidance. A trajectory is generated using RRT* between specified start and goal points. Yellow circles are dynamic obstacles that follow a random walk. Cyan crosses mark locations where obstacles are detected by the agent.

Table 1 summarizes the results of experiments that varied two parameters: η (the RRT* extend distance) and $\frac{l_{\text{waypoint}}}{l_{\text{traj}}}$ (the look-ahead ratio). The extend distance η influences the smoothness of the generated path, while the look-ahead ratio affects how far ahead the agent plans along the path. A larger look-ahead ratio can smooth out the trajectory but may also increase the risk of getting stuck on obstacles. Conversely, a smaller ratio causes the agent to follow the path more closely, potentially improving obstacle clearance at the cost of trajectory smoothness.

For each parameter setting, 100 tests were conducted with randomized start and goal locations. The start and goal seeds were initialized to 0 and 101, respectively, and incremented by 1 in each test to ensure consistency across test cases.

Parameter		Success Rate	Failure Rate (MPPI)	Failure Rate (RRT)
η	0.5	.40	.50	.10
	0.75	.43	.44	.13
	1.0	.41	.43	.16
$\frac{l_{waypoint}}{l_{traj}}$	1	.39	.46	.15
	1.5	.40	.45	.15
	2	.45	.41	.14
	3	.42	.41	.17

Table 1. Combined detailed outcome breakdown for different parameters

The success rate was defined as the percentage of trials in which the agent successfully reached the goal. Failure cases were categorized based on whether they occurred during RRT* path planning or MPPI trajectory following. Results indicate that overall success was achieved in approximately 40% of trials. Most failures (45%) occurred during the MPPI trajectory-following phase, while 15% were due to the RRT* planner. Variations in the tested parameters had minimal impact on overall success or failure rates.

5. Discussion and conclusions

The previous section highlights significant room for improvement, as all parameter variations resulted in a success rate of only around 40%. This limitation is partly due to known issues with RRT-style planners, which can struggle to find paths through narrow passages. Some improvement in failure rates may be achieved by increasing the maximum number of samples or by adopting bi-modal sampling, where paths are generated simultaneously from both the start and goal points. Alternatively, switching to a different planning technique, such as A*, could also enhance performance.

However, updates to the global planning algorithm alone are unlikely to yield major improvements. Greater attention should be given to failures during the trajectory-following phase. A key issue arises when waypoints are sampled in regions that the agent cannot physically traverse. Figure 4 shows an example where the planner successfully finds a path through a narrow gap, but the agent becomes stuck at the entrance due to insufficient clearance. This limitation is partially influenced by the Control Barrier Function (CBF), and reducing the associated margin could allow the agent to get closer to walls. Still, certain areas are fundamentally too narrow for the agent to fit through. Excluding such regions from test sampling would improve results but would require manual inspection and modification of every map.

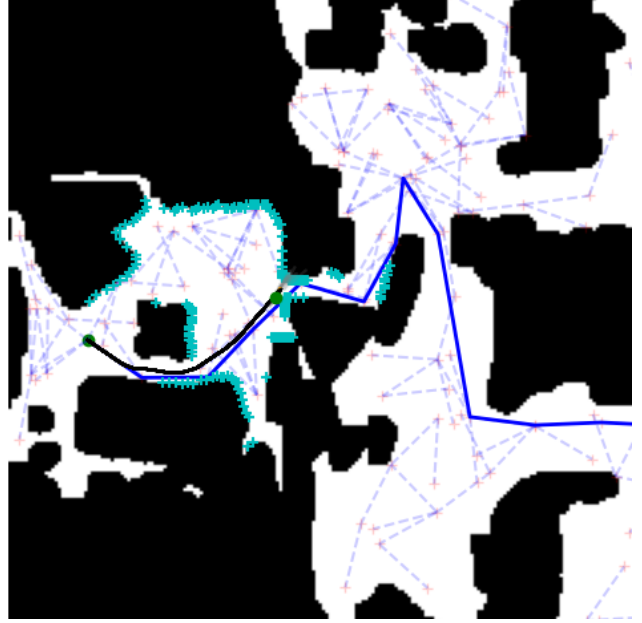


Figure 4. Example of a failure where the path attempts to take the agent through a gap that is too narrow. The agent gets stuck at the entrance as it cannot make any progress.

Further improvements could be made by adjusting the MPPI cost function. Currently, the agent follows a moving waypoint, and trajectories are generated based on the terminal cost associated with that point. As a result, the agent may not align with the RRT* path and instead head toward a changing target, potentially causing it to cut corners around obstacles and become stuck. Modifying the cost function to minimize the deviation between MPPI trajectories and the full RRT* path would encourage adherence to a feasible path and reduce the likelihood of such failures.

In this work, we presented a robust and responsive framework for autonomous vehicle control by integrating Model Predictive Path Integral (MPPI) control with Control Barrier Functions (CBFs). Our approach addresses key challenges in real-world navigation, including uncertain dynamics and interactive multi-agent behavior. Experimental results show that MPPI with CBF performs well in static environments, but trajectory-following in dynamic contexts—especially with narrow or infeasible regions—remains difficult. While minor improvements may be achieved through global planning enhancements, our findings suggest that the most promising gains lie in refining local trajectory tracking and improving alignment between planners. This work lays a strong foundation for scalable, safe autonomous driving and points to promising directions for future improvements in sampling, cost modeling, and multi-agent coordination under uncertainty.

6. Future Work

6.1. MPPI improvements

On the implementation side, Algorithm 1 was developed using both the standard NumPy library and the parallel-capable JAX framework. However, due to version incompatibilities between cuDNN and JAX, the GPU-accelerated implementation is currently non-functional. As of this writing, the parallelized version runs only on the CPU. We aim to resolve this issue to enhance sampling efficiency, enabling broader exploration of MPPI improvements.

Another direction for future work is refining the sampling strategy in MPPI. The current baseline occasionally exhibits jittery motion or unexpected reversals near the goal. These instabilities may be mitigated by incorporating motion smoothing techniques such as filtering. On the algorithmic side, we also plan to investigate output-sampled MPPI (o-MPPI) [15] and explore data-driven methods for modeling inverse dynamics.

6.2. MPPI with Game Theoretic Approach

In multi-agent driving scenarios, vehicles need to reason not only about their own trajectories but also about the actions of others. [5] proposes a game-theoretic framework that models this interaction as a generalized Nash equilibrium (GNE), where each agent optimizes its own objective under shared constraints like collision avoidance. To handle uncertainty, the problem is reformulated as a stochastic potential game and solved as a single stochastic optimal control problem (SOCP), capturing the coupled behavior of all agents under noise and dynamic environments.

While their solution uses traditional MPC solvers, our goal is to apply this formulation within the MPPI framework. MPPI's sampling-based nature makes it well-suited for solving the resulting SOCP in real time, especially under nonlinear dynamics and uncertainty. Integrating this game-theoretic structure into MPPI could enable agents to plan strategically in interactive, stochastic settings while maintaining fast sampling-based control.

References

- [1] *Global Status Report on Road Safety 2023*. World Health Organization, 1st ed edition. 1
- [2] Adriano Alessandrini, Andrea Campagna, Paolo Delle Site, Francesco Filippi, and Luca Persia. Automated vehicles and the rethinking of mobility and cities. 5:145–160. 1
- [3] Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Genaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European control conference (ECC)*, pages 3420–3431. Ieee, 2019. 1
- [4] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning, 2011. 2
- [5] Michael Khayyat, Saverio Bolognani, Stefano Arrigoni, and Francesco Braghin. A sampling-based approach to urban motion planning games with stochastic dynamics. pages 1–15. 6
- [6] S. LAVALLE. Rapidly-exploring random trees : a new tool for path planning. *Research Report 9811*, 1998. 2
- [7] Simon Le Cleac’h, Mac Schwager, and Zachary Manchester. ALGAMES: a fast augmented lagrangian solver for constrained dynamic games. 46(1):201–215. 1
- [8] Dakota McCarty and Hyun Woo Kim. Risky behaviors and road safety: An exploration of age and gender influences on road accident rates. 19(1):e0296663. 1
- [9] Ihab S Mohamed, Junhong Xu, Gaurav S Sukhatme, and Lantao Liu. Towards efficient mppi trajectory generation with unscented guidance: U-mppi control strategy. *IEEE Transactions on Robotics*, 2025. 1
- [10] Tajammul Pangarkar. Autonomous vehicles statistics 2025 by type, technology, driving, 2025. 1
- [11] Sebastian Thrun et al. Stanley: The robot that won the DARPA grand challenge. 23(9):661–692. 1
- [12] Chris Urmson et al. Tartan racing: A multi-modal approach to the DARPA urban challenge. 1
- [13] Johannes Weyer, Robin D. Fink, and Fabian Adelt. Human-machine cooperation in smart cars. an empirical investigation of the loss-of-control thesis. 72:199–208. 1
- [14] Grady Williams, Paul Drews, Brian Goldfain, James M Rehg, and Evangelos A Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1433–1440. IEEE, 2016. 1
- [15] Leon Liangwu Yan and Santosh Devasia. Output-sampled model predictive path integral control (o-mppi) for increased efficiency. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14279–14285. IEEE, 2024. 1, 6
- [16] Ji Yin, Charles Dawson, Chuchu Fan, and Panagiotis Tsiontras. Shield model predictive path integral: A computationally efficient robust mpc approach using control barrier functions. *arXiv preprint arXiv:2302.11719*, 2023. 1, 3